LIFELINK

A BLOOD DONATION AND DONOR REQUEST MANAGEMENT SYSTEM

PROJECT REPORT

Submitted by

HARSHAVARDHINI M – 241701018

SANDHIYA E – 241701048

THAMIZHINI R – 241701060

in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND DESIGN

RAJALAKSHMI ENGINEERING COLLEGE – THANDALAM

ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this report titled “LIFELINK: A BLOOD DONATION AND DONOR REQUEST MANAGEMENT SYSTEM” is the bonafide work of HARSHAVARDHINI M, SANDHIYA E and THAMIZHINI R, carried out under the supervision.

The project focuses on simplifying the process of connecting blood donors with people who need blood by providing a digital platform for user registration, donor profiles, blood-request creation and request visibility. The implementation uses Flutter for the application interface and Firebase services for authentication and cloud data storage, as configured in the project.

HARSHAVARDHINI M – 241701018

SANDHIYA E – 241701048

THAMIZHINI R – 241701060

Signature of the Head of the Department	**Signature of the Project Guide**
Dr. S. Vinodh Kumar	Dr. P. Revathy
Professor and Head of the Department	Professor
Department of Computer Science and Design	Department of Computer Science and Design
Rajalakshmi Engineering College	Rajalakshmi Engineering College

Submitted for the project review / viva-voce examination held on: _____

Internal Examiner	**External Examiner**
Signature: _____	Signature: _____

DECLARATION

We hereby declare that the project report entitled “LifeLink: A Blood Donation and Donor Request Management System” is prepared by our team as part of our academic project work. The report describes the design and implementation undertaken by the team. Any external material used for background or technical explanation is acknowledged in the references section.

Place: Chennai

Date: 02-11-2026

Signatures of the team Members :

Sandhiya E

Harshavardhini M

Thamizhini R

ABSTRACT

LifeLink is a mobile application project intended to support blood donation coordination by connecting registered users, donors and people who need blood through a single digital interface. In conventional arrangements, locating a suitable donor and communicating a blood requirement may depend on separate phone calls, personal contacts or informal messaging groups. This can make information difficult to organise and requests harder to track. LifeLink aims to provide a more structured way to register users, maintain donor profiles and create or view blood requests.

The application interface is developed using Flutter, enabling a consistent user experience across supported devices. Firebase services are used for user authentication and cloud-based data management, according to the project configuration. Role-based screens separate the user experience for donors and requesters, while request information is intended to be displayed to the relevant users through profile and request views. The project also includes interface considerations such as readable text, navigation between screens and support for light and dark themes.

The current implementation provides a foundation for digital donor-request coordination. Testing focuses on registration and login flows, profile navigation, request creation, donor-side request visibility, and the reliability of data exchange with Firebase. The system is not a substitute for clinical assessment, blood-bank procedures or emergency medical services. Further work can improve request matching, notifications, verification, privacy controls and testing with real users.

Keywords: Blood donation, donor management, Flutter, Firebase, mobile application, blood request, healthcare support.

ACKNOWLEDGEMENT

We express our sincere gratitude to the management of [Rajalakshmi Engineering College] for providing the facilities and support required to carry out this project. We thank our Principal, Head of the Department, project coordinator and project guide for their guidance, feedback and encouragement throughout the development process. We also thank our faculty members and peers who provided suggestions during the design, implementation and review stages. Finally, we thank everyone who helped us understand the practical challenges involved in blood donation coordination and the importance of making donor information and blood requests easier to manage.

SANDHIYA E – 241701048

HARSHAVARDHINI M – 241701018

THAMIZHINI R – 241701060

TABLE OF CONTENTS

Section	Title	Page
Preliminary	Bonafide Certificate	ii
	Declaration	iii
	Abstract	iv
	Acknowledgement	v
	Table of Contents	vi
	List of Tables	vii
	List of Figures	viii
	List of Abbreviations	ix
1	Introduction	1
2	Literature Review and Existing Systems	5
3	System Analysis and Design	7
4	Methodology and Implementation	11
5	Testing and Results	15
6	Conclusion and Future Enhancements	18
	Appendices	19
	References	23

LIST OF TABLES

Table 1.1 Limitations of the existing system 3

Table 2.1 Comparison with existing approaches 6

Table 3.1 Functional requirements 7

Table 3.2 Non-functional requirements 7

Table 3.3 User roles 8

Table 3.4 Data entities and representative fields 9

Table 3.5 Use cases 9

Table 4.1 Technology stack 11

Table 4.2 Application modules 11

Table 5.1 Test cases 15

Table 5.2 User feedback summary 16

Table 6.1 Future enhancements 19

LIST OF FIGURES

Figure A.1 Priority donors screen 20

Figure A.2 Patient dashboard screen 21

Figure A.3 Blood bank profile screen 22

LIST OF ABBREVIATIONS

Abbreviation	**Meaning**
UI	User Interface
UX	User Experience
DB	Database
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
OTP	One-Time Password
SDK	Software Development Kit
IDE	Integrated Development Environment
OS	Operating System
UAT	User Acceptance Testing
RBAC	Role-Based Access Control
GDPR	General Data Protection Regulation (general privacy reference)

CHAPTER 1 – INTRODUCTION

1.1 General

Blood donation is an important part of healthcare systems, and timely access to donor information can help people coordinate their search for blood. In many communities, requests are circulated through personal networks, phone calls and messaging groups. These approaches can work, but the information may be scattered, repeated or difficult to update.

LifeLink is designed as a mobile application that organises basic donor and blood-request information in one place. It provides user authentication, profile-based access and screens intended to help users create and view blood requests. By using a familiar mobile interface and cloud-based data storage, the project seeks to make the process more structured and accessible.

1.2 Background

Mobile applications and cloud databases are widely used to manage user-generated information. Flutter supports building interactive interfaces from a shared codebase, while Firebase offers authentication and database services that can be integrated into an application. Combining these technologies provides a practical foundation for a student project that needs user accounts and data shared between devices.

1.3 Problem Statement

People seeking blood may face difficulty identifying potential donors and communicating the details of a request in an organised way. Existing informal methods do not always provide a central place to maintain donor details, create requests and view relevant request information. LifeLink addresses this problem by developing a mobile application that supports registration, donor profiles and blood-request management.

1.4 Motivation

The motivation for LifeLink is to use accessible mobile technology to support community-level donor coordination. A clear application flow can reduce dependence on scattered messages and help users find request information in a more organised format. The project also provides practical experience in mobile UI design, authentication, cloud data storage and role-based application logic.

1.5 Objectives

- To develop a mobile application interface using Flutter.
- To support user registration and login using Firebase Authentication, where configured.
- To maintain user profile information and distinguish relevant user roles such as donor and requester.
- To provide a structured form for creating blood requests.
- To display relevant blood-request information in the appropriate user profile or dashboard.
- To connect the application interface with cloud-based data storage.
- To design clear navigation and readable interfaces, including light and dark theme support.
- To test the main application flows and identify issues that need further improvement.

1.6 Scope

1.6.1 Included Scope

- User registration and login screens.
- Profile or dashboard views based on the user flow.
- Donor-related profile information.
- Creation and display of blood requests.
- Firebase integration for authentication and cloud data, according to the implemented configuration.
- Application navigation, theme styling and basic usability improvements.

- Functional testing of major screens and data flows.

1.6.2 Limitations

- The application does not verify medical eligibility or guarantee the availability of blood.
- The accuracy of user-provided donor information depends on the information entered and any verification process implemented.
- Request visibility depends on correct database writes, queries, role information and Firebase security rules.
- Notifications, location-based matching, hospital integration and identity verification should be treated as future enhancements unless confirmed in the final build.
- The app must not replace emergency medical services or official blood-bank procedures.

1.7 Existing System

In an informal donor-search process, a requester may contact friends, relatives, community groups or social media networks. Details such as blood group, location, urgency and contact information can be shared across several channels. This can lead to repeated messages, outdated information and difficulty tracking which requests have been fulfilled.

1.8 Limitations of Existing System

Table 1.1 Limitations of the existing system

Limitation	Description
Scattered information	Donor and request details may be distributed across multiple communication channels.
Manual coordination	Users may need to contact many people individually.
Difficult tracking	It can be difficult to determine whether a request is still active or has been fulfilled.
Limited structure	Requests may not follow a consistent format for blood group, location and contact details.
Information freshness	Old or incomplete posts may continue to circulate.

1.9 Proposed System

LifeLink proposes a mobile-based interface in which users can sign in, access profile-related screens and work with blood-request information through a more consistent flow. Flutter is used for the UI, and Firebase is used for authentication and cloud data handling as implemented. The application separates screen-level presentation from the underlying data operations so that information can be stored and retrieved in a structured manner.

1.10 Advantages of Proposed System

- Centralised application flow for donor and request information.
- Consistent mobile interface built with Flutter.
- Cloud-backed authentication and data storage through Firebase, where configured.
- Role-oriented screens to support different user tasks.
- Potential to extend the system with notifications, verification and improved matching.

CHAPTER 2 – LITERATURE REVIEW AND EXISTING SYSTEMS

2.1 Introduction

This chapter reviews the technology concepts and application patterns relevant to LifeLink. The project draws on mobile application development, cloud authentication, database-backed forms and user-centred interface design. The review is technology-oriented; a final academic submission should add verified research papers and local studies selected by the project team.

2.2 Mobile Applications for Community Coordination

Mobile applications can provide a consistent interface for collecting, storing and retrieving information. For community coordination tasks, useful design features include clear forms, searchable records, status information and contact details. A structured request form can help ensure that key information is entered in a predictable format.

2.3 Flutter

Flutter is a UI toolkit used to build applications from a shared codebase. Its widget-based approach supports reusable interface components, responsive layouts, navigation and theme configuration. In LifeLink, Flutter is used to create screens such as login, registration, dashboards, profiles and request-related views.

2.4 Firebase

Firebase provides backend services commonly used by mobile applications. Firebase Authentication can manage user sign-in, while Cloud Firestore or another configured Firebase database can store profile and request records. The exact database service, collection names and security rules should be documented from the final project configuration.

2.5 User-Centred Design and Accessibility

A community-support application should use clear labels, understandable navigation and consistent feedback. Text contrast must remain readable in both light and dark themes. Forms should identify required fields and explain errors in plain language.

The interface should avoid implying that a donor is medically eligible or that a request has been fulfilled unless that status has been confirmed.

2.6 Comparison with Existing Approaches

Table 2.1 Comparison with existing approaches

Approach	Strengths	Limitations
Phone calls and personal contacts	Direct communication and familiar to users.	Manual, difficult to scale and dependent on personal networks.
Messaging groups / social media	Fast sharing and broad reach.	Requests can be buried, duplicated or become outdated.
Spreadsheet or static directory	Can organise structured records.	May require manual updates and does not inherently provide role- based mobile workflows.
LifeLink proposed application	Structured mobile flow for profiles and requests.	Effectiveness depends on data quality, backend configuration, user adoption and further testing.

2.7 Research / Development Gap

The project focuses on the practical integration gap between user registration, donor profiles and request handling in a single student-developed mobile application. Its contribution is an implemented workflow rather than a claim of a new medical or matching algorithm. More advanced matching, validation and notifications remain areas for future development.

2.8 Summary

The reviewed concepts support the choice of Flutter for the interface and Firebase for account and data services. The next chapter describes the requirements and architecture used to organise the LifeLink implementation.

CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN

3.1 General

System design identifies the users, information and application modules before implementation. LifeLink is organised around authentication, profile management, blood-request entry, request retrieval and user feedback. Each screen interacts with the data layer through application logic.

3.2 System Requirements

3.2.1 Functional Requirements

Table 3.1 Functional requirements

ID	Requirement
FR1	The application shall provide registration and login interfaces.
FR2	The application shall authenticate users through the configured authentication service.
FR3	The application shall collect and display profile information.
FR4	The application shall support donor-related profile information where applicable.
FR5	The application shall allow a user to submit a blood request using a structured form.
FR6	The application shall store request data in the configured Firebase data store.
FR7	The application shall retrieve and display relevant request records on the appropriate screen.
FR8	The application shall provide navigation between the main screens.
FR9	The application shall provide clear feedback for validation, loading and failure states.
FR10	The application shall apply consistent styling and readable text in supported themes.

3.2.2 Non-Functional Requirements

Table 3.2 Non-functional requirements

Requirement	Description
Usability	Screens should be easy to understand and navigation should be consistent.
Reliability	Data should be written and read correctly, with errors handled clearly.
Security	Access to profile and request records should follow appropriate authentication and database rules.

Maintainability	UI components and data operations should be organised so changes can be made without rewriting the whole application.
Performance	Common screens and database operations should respond within a reasonable time on supported devices.
Accessibility	Text contrast, labels and interactive targets should remain usable across themes and screen sizes.

3.3 User Roles

Role / user type Primary activities

Table 3.3 User roles

Registered user / requester	**Sign in, maintain profile information and submit a blood request.**
Donor	Maintain donor-related profile information and view relevant blood-request information.
Administrator (if implemented)	Manage or review records only if an administrative role exists in the actual application.

3.4 Proposed Architecture

The logical architecture contains four main layers: (1) presentation layer, consisting of Flutter screens and widgets; (2) application logic layer, which validates form input and manages navigation; (3) authentication and data-access layer, which communicates with Firebase services; and (4) cloud data layer, which stores user and blood-request records. The UI displays success, empty, loading and error states based on the results returned by the data layer.

USER → FLUTTER UI → VALIDATION / APPLICATION LOGIC → FIREBASE AUTHENTICATION + DATABASE → RESULT SHOWN IN UI

3.5 Data Flow

- The user opens the application and selects registration or login.
- Authentication details are validated and submitted to the configured authentication service.
- After successful sign-in, the application loads profile information and presents the relevant dashboard.

- A requester enters blood-request details in the request form.
- The application validates the fields and writes the request record to the database.
- The donor-side screen queries the relevant request records and displays the results.
- The user receives a confirmation, empty-state message or error message depending on the outcome.

3.6 Database Design

The following is a conceptual schema to help document the application. Replace the example field names with the actual collection/document structure used in the project.

Table 3.4 Data entities and representative fields

Entity / collection	Representative fields	Purpose
Users	uid, name, email, role, phone, bloodGroup, location, createdAt	Stores account-linked profile details.
bloodRequests	requestId, requesterId, patientName or label, bloodGroup, units, location, contact, urgency, status, createdAt	Stores requests submitted by users.
notifications (optional)	notificationId, recipientId, requestId, message, read, createdAt	Can support future or implemented notification workflows.

Important: Do not store unnecessary sensitive medical information. Apply database rules that restrict reads and writes to authenticated users and the access pattern required by the application. The exact rule set should be reviewed against the actual Firebase configuration.

3.7 Use Cases

Table 3.5 Use cases

Use case	Actor	Outcome
Register account	New user	An account is created if the submitted information is valid.
Log in	Registered user	The user reaches the appropriate application screen.
View profile	Registered user	The user sees their stored profile information.
Create blood request	Requester	A validated request is saved to the data store.
View requests	Donor / authorised user	Relevant stored requests are displayed.

Sign out	Registered user	The session ends and the user returns to the authentication screen.

3.8 User Interface Design

The interface should maintain consistent spacing, button styles, input labels and navigation. Login and registration screens should show validation feedback. Dashboards should prioritise the actions most relevant to the signed-in user. Request cards should display the important request fields without overcrowding the screen. In dark mode, foreground text, icons, input labels, hint text and card backgrounds must use theme-aware colours rather than fixed light-mode values.

3.9 Security and Privacy Considerations

- Do not place service-account credentials or private keys inside the mobile application.
- Use Firebase Authentication and restrictive database security rules.
- Validate important fields on the application side and enforce access control in database rules.
- Collect only information needed for the donor-request workflow.
- Explain to users what profile and request details are visible to other users.
- Avoid claiming that donor details are verified unless a verification procedure exists.

CHAPTER 4 – METHODOLOGY AND IMPLEMENTATION

4.1 General

The project methodology follows a staged process: problem identification, requirement analysis, interface planning, technology selection, implementation, integration and testing. This sequence supports incremental development and makes it easier to isolate issues when a screen or data operation does not behave as expected.

4.2 Technology Stack

Table 4.1 Technology stack

Component	Technology	Role in LifeLink
Mobile UI	Flutter / Dart	Builds screens, widgets, forms, navigation and theme support.
Authentication	Firebase Authentication (if enabled)	Supports account creation and sign- in.
Cloud data	Firebase database configured by the project	Stores profile and blood-request records. Confirm whether this is Cloud Firestore or Realtime Database.
Development environment	Visual Studio Code / Android Studio	Used to edit, run and debug the application.
Testing device	Android phone / emulator / supported browser	Used to verify navigation, UI and data operations.

4.3 Application Modules

Table 4.2 Application modules

Module	Description
Authentication module	Handles registration, login, sign-out and session-aware navigation.
Profile module	Displays and manages user details relevant to the signed- in role.
Donor module	Provides donor-specific information and the donor-side view of requests, as implemented.
Blood request module	Collects request details, validates input and saves the request.
Request listing module	Retrieves and displays request records to the relevant users.
Theme and UI module	Maintains common styles, readable text and light/dark theme behaviour.
Firebase integration module	Connects UI actions to authentication and database reads/writes.

4.4 Registration and Login Flow

The user enters the required details in the registration or login form. The application validates the input before making the authentication request. When authentication succeeds, the application reads the user profile and determines the next screen based on the role or profile fields used by the implementation. When authentication fails, a clear error message should be shown and the user should remain able to correct the input.

4.5 Blood Request Creation Flow

The requester opens the request form and enters the required details, such as blood group, location, contact information and any additional fields implemented in the project. The form validates mandatory values and submits a structured record to Firebase. After a successful write, the application confirms that the request was submitted. If the write fails, the application should show an error and avoid falsely reporting success.

4.6 Donor Request Retrieval Flow

The donor-side screen should query the same database location in which the request form writes the request record, then map returned documents into the request-card model and display them. If no records are returned, the interface should distinguish an actual empty result from a loading or query failure. The query, collection name, field names, authentication state and Firebase rules must agree across the create and read paths.

4.7 Common Integration Considerations

- Use consistent collection names and field keys in create and read operations.
- Confirm that the user role is saved and read using the same spelling and case.
- Ensure the donor screen queries the correct collection and does not filter out valid records unintentionally.
- Handle asynchronous reads with loading, success, empty and error states.
- Check Firebase security rules using the actual signed-in user rather than weakening rules for convenience.

- Keep theme colours and text styles in a central theme configuration wherever possible.

4.8 Development Workflow

- Set up the Flutter project and connect Firebase configuration.
- Create reusable styles and navigation routes.
- Implement authentication screens and session handling.
- Implement profile and role-specific dashboard screens.
- Implement blood-request form and database write operation.
- Implement donor-side request retrieval and display.
- Review theme contrast and layout on the target device.
- Run functional tests, fix defects and prepare the final APK or build.

4.9 Feasibility Study

4.9.1 Technical Feasibility

The project uses commonly available mobile development tools and a cloud backend. Flutter and Firebase provide libraries and documentation suitable for student-level implementation. The main
technical risks are correct Firebase configuration, consistent data modelling, access-control rules and reliable handling of asynchronous operations.

4.9.2 Economic Feasibility

The development toolchain can be used at low cost, and a basic prototype can be built with free development resources. Actual operating costs may depend on Firebase usage, hosting choices, storage volume and any third-party services used.

4.9.3 Operational Feasibility

A mobile interface is familiar to many users. Operational success depends on clear forms, accurate information, reliable connectivity, privacy safeguards and a process for keeping donor and request details up to date.

4.10 Ethical Considerations

LifeLink deals with information that may be personal and health-adjacent. Users should understand how their details and requests are shared. The project should not present the application as a medical authority, promise that a donor is available, or replace professional blood-bank coordination. User feedback should be collected with consent and reported honestly.

CHAPTER 5 – TESTING AND RESULTS

5.1 General

Testing verifies whether the implemented application behaves as expected for the main user flows. Tests should be carried out on the final build and recorded with the actual result, status and evidence. The table below is a test plan template; fill the result column only after executing each test on the current version.

5.2 Testing Methodology

5.2.1 Unit Testing

Individual functions such as form validation, data-model conversion and request-card formatting can be checked independently using valid and invalid inputs.

5.2.2 Integration Testing

Integration testing checks that Flutter screens communicate correctly with Firebase Authentication and the configured database. Particular attention should be given to the relationship between request creation and donor-side request retrieval.

5.2.3 Functional Testing

Functional tests compare the observed behaviour with the requirements. A test is successful only when the actual behaviour matches the expected result on the tested build.

5.3 Test Cases

Table 5.1 Test cases

Test ID	Test scenario	Expected result	Actual result / status
TC01	Register with valid details	Account is created and appropriate next screen opens.	[Fill after testing]
TC02	Register with missing/invalid details	Validation or authentication error is shown.	[Fill after testing]
TC03	Log in with valid credentials	User is authenticated and reaches the correct screen.	[Fill after testing]
TC04	Log in with invalid credentials	Login is rejected and an understandable message appears.	[Fill after testing]

TC05	Open profile screen	Profile information is loaded and displayed correctly.	[Fill after testing]
TC06	Submit a valid blood request	Request record is stored and success feedback appears.	[Fill after testing]
TC07	Submit request with required field missing	Form prevents invalid submission or displays validation.	[Fill after testing]
TC08	Open donor request list after request submission	Relevant request appears on the donor screen.	[Fill after testing]
TC09	Open request list when there are no records	A clear empty-state message appears.	[Fill after testing]

Table 5.2 User feedback summary

TC10	**Simulate database/network error**	**The application shows an error state and remains usable.**	**[Fill after testing]**
TC11	Switch to dark mode	Text, icons, inputs and cards remain readable.	[Fill after testing]
TC12	Sign out and reopen protected screen	Session ends and protected content is not shown as authenticated.	[Fill after testing]

5.4 Initial Results

The project implementation establishes the core application structure and the main user flows for a blood donation coordination app. The application is built around Flutter screens and Firebase integration. The final report should include screenshots showing the actual login/registration page, dashboard, donor profile, blood request form, donor request list and light/dark theme if these are present in the delivered build.

Do not report numerical performance, user satisfaction percentages or successful test counts unless they were measured and documented. The team should insert actual test outcomes and any userfeedback results collected for the project.

5.5 Known Issues and Debugging Focus

A key integration issue to check in the final build is whether submitted blood requests appear in the donor profile. Debugging should compare the exact database collection or path used by the write operation with the query used by the donor screen, then verify field names, role values, authentication state, asynchronous

refresh behaviour and database rules. UI theme testing should also confirm that text colours are theme-aware and that no fixed white backgrounds or red text create poor contrast in dark mode.

5.6 User Feedback

If feedback forms were collected from three users, summarise their responses here using the actual submitted forms. Include the number of participants, the questions asked, the common positive observations, reported difficulties and suggested improvements. Do not invent participant responses or present informal feedback as a statistically representative study.

Feedback area	**What to record**
Ease of use	Whether users could navigate without help.
Clarity	Whether labels and request information were understandable.
Functionality	Whether the main flows worked as expected during the session.
Visual design	Feedback on layout, readability and theme appearance.
Improvements	Specific changes suggested by participants.

CHAPTER 6 – CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

LifeLink is a mobile application project aimed at making blood donor and request information easier to organise. By combining a Flutter-based interface with Firebase services, the project provides a foundation for account access, profile-oriented screens and structured blood-request handling. The central goal is to provide a more consistent digital workflow than relying entirely on scattered messages and manual coordination.

The reliability of the final system depends on correct authentication, consistent database schemas, suitable security rules and clear handling of loading, empty and error states. Testing should therefore focus not only on individual screens but also on the complete path from creating a request to displaying it to the intended users. The application should be presented as a coordination aid, not as a substitute for medical advice, blood-bank procedures or emergency services.

6.2 Current Project Status

Area	**Status to confirm before submission**
Flutter interface	[List completed screens in the current build]
Authentication	[Confirm registration, login and sign-out]
Profile and role flow	[Confirm role values and destination screens]
Blood request creation	[Confirm request write to Firebase]
Donor-side request display	[Confirm retrieval and display in the donor profile]
Theme support	[Confirm light/dark mode readability]
Testing	[Insert executed test results]

6.3 Limitations

- Request and donor data may depend on user-entered information and may not be independently verified.
- The app requires a compatible device and network access for cloud operations.
- Advanced donor matching and notification functions should not be claimed unless implemented and tested.
- The system does not establish medical eligibility or guarantee blood availability.

- Further security, usability and real-world testing is required before wider deployment.

6.4 Future Enhancements

Enhancement	Description
Verified donor profiles	Introduce a suitable verification process and show verification status transparently.
Search and filtering	Filter available request information by blood group, area and request status.
Push notifications	Notify relevant users about new requests, subject to consent and notification settings.
Request status tracking	Allow authorised users to mark requests as open, in progress or fulfilled.

Table 6.1 Future enhancements

Improved matching	**Rank relevant requests and donors using appropriate, explainable criteria.**
Privacy controls	Let users manage visibility of contact and profile information.
Accessibility and localization	Improve screen-reader labels, contrast, text scaling and language support.
Analytics and evaluation	Measure task completion, response times and usability using a documented test protocol.

6.5 Final Remarks

LifeLink demonstrates how mobile application development and cloud services can be combined to support a community-oriented use case. With careful testing, privacy-aware design and continued refinement, the project can be developed into a more reliable platform for coordinating donor information and blood requests.

APPENDICES

APPENDIX A – PROJECT SCREENSHOTS

The following screenshots are reproduced from the supplied report. Each figure is captioned below.

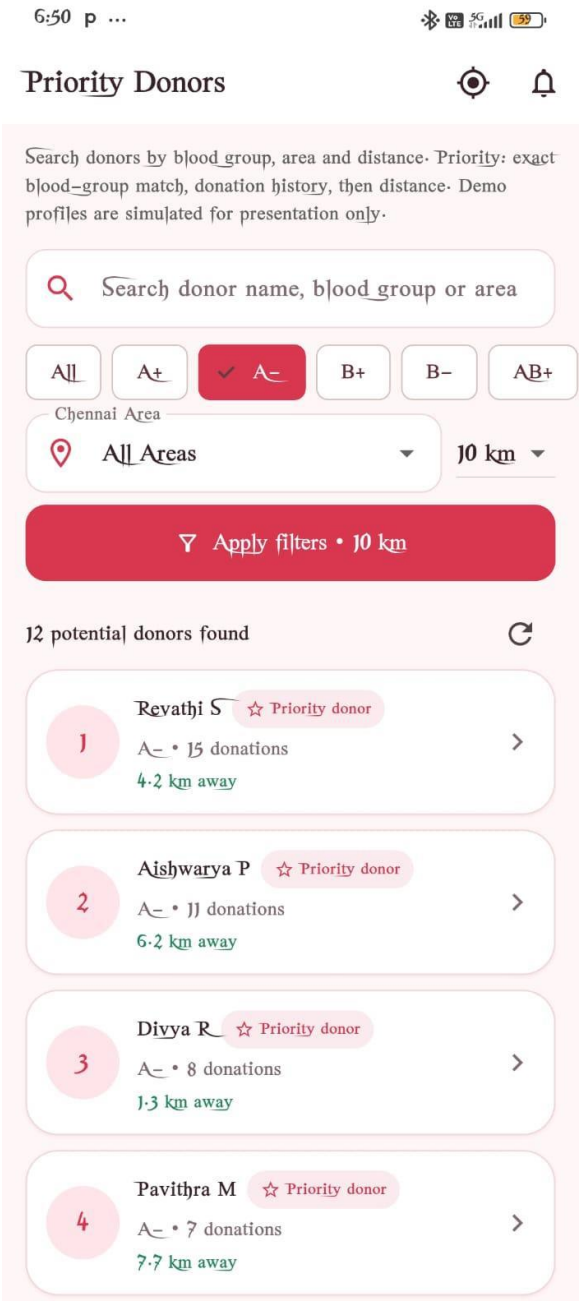


Figure A.1 – Priority donors screen

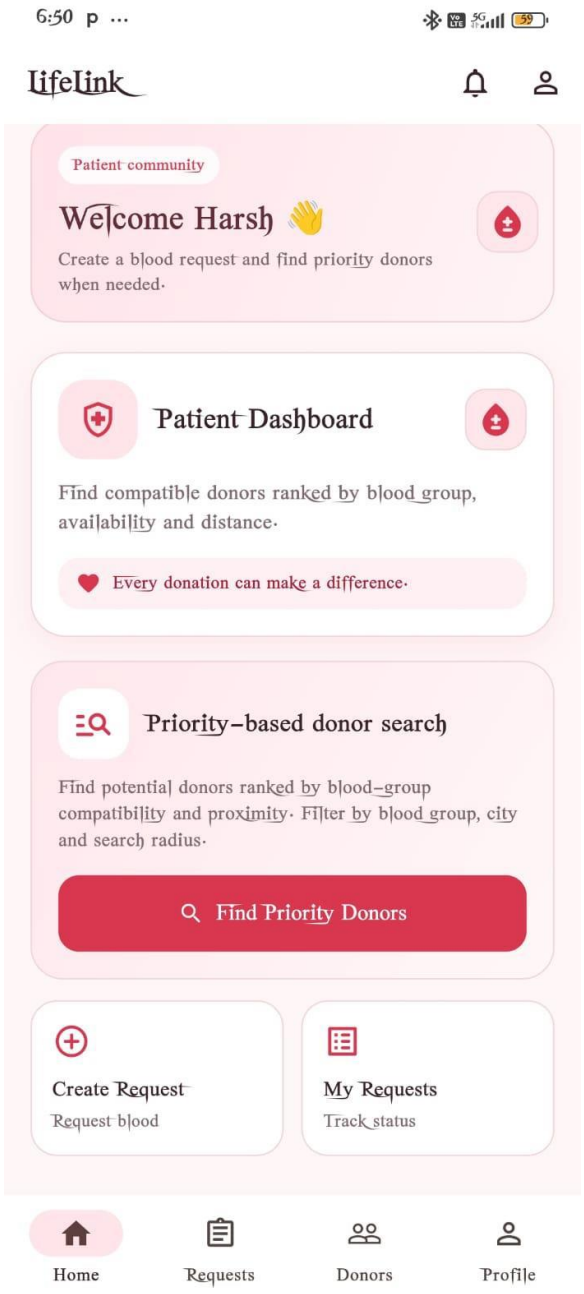


Figure A.2 – Patient dashboard screen

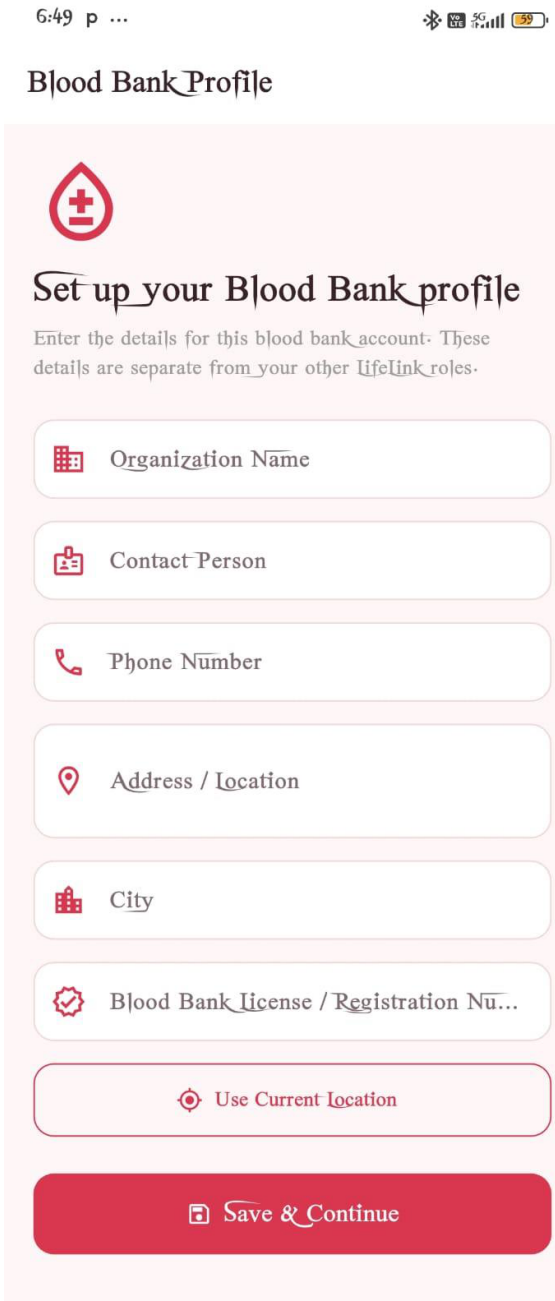


Figure A.3 – Blood bank profile screen

APPENDIX B – USER FEEDBACK SUMMARY

Insert the anonymised summary of the three user feedback forms, if collected. Do not include personal phone numbers, email addresses or other identifying details. Report the actual answers and distinguish user suggestions from changes already implemented.

APPENDIX C – SUBMISSION CHECKLIST

☐ Replace every square-bracket placeholder with accurate team and institution details.

☐ Confirm the actual Firebase database service and schema.

☐ Remove features and roles that are not present in the final build.

☐ Run every test case and record actual outcomes.

☐ Insert screenshots from the final application.

☐ Add verified research references in the required institutional format.

☐ Update the table of contents and page numbers after formatting.

☐ Review the report with the project guide before submission.

APPENDIX D – DETAILED REQUIREMENTS AND USE-CASE SPECIFICATION

D.1 Purpose of the Appendix

This appendix expands the requirements discussed in Chapter 3. It provides a structured reference for reviewing the expected behaviour of LifeLink. The descriptions are written as system requirements and use-case specifications. They should be checked against the final application before being treated as implemented functionality. A requirement describes what the application is expected to do; it does not by itself prove that the feature is available in the submitted build.

D.2 Stakeholders

The main stakeholders are a person requesting information about blood availability, a person who is willing to donate, the project development team, and the institution reviewing the project. A requester needs a clear way to enter the blood group and location-related details of a request. A donor needs understandable information so that they can decide whether to contact the requester. The development team needs consistent requirements to guide interface design, data handling and testing.

The project guide and reviewers need a report that distinguishes implemented functions from proposed enhancements.

D.3 User Registration Use Case

Primary actor: new application user. Preconditions: the application is installed or opened and the user can access the registration screen. Main flow: the user opens registration, enters the required fields, reviews the details and submits the form. The application checks that mandatory fields are present and that input is in an acceptable format. If the authentication service is configured and reachable, the application submits the account request and displays an appropriate success or error message.

Alternative flows include missing fields, invalid email format, weak or mismatched password, an already registered account, unavailable network access and an authentication service error. Postcondition: an account is created only when the authentication service confirms success. The interface should not display a success message before confirmation.

D.4 Login Use Case

Primary actor: registered user. Preconditions: the user has an account and the login screen is available. Main flow: the user enters the registered credentials and submits the form. The application validates the fields, requests authentication and waits for the service response. On success, the user is directed to the appropriate starting screen according to the application flow. On failure, a clear message is displayed without exposing technical credentials or internal service details.

Alternative flows include an empty email field, an empty password field, incorrect credentials, a disabled account if such a state is supported, and a network failure. A password should never be displayed as ordinary text by default.

D.5 Profile and Role Use Case

A profile screen presents information associated with the signed-in user. Where the project uses separate donor and requester experiences, the role value must be stored and read consistently. The role name, field key and accepted values should be documented in one place. If a role is missing or unrecognised, the application should avoid routing the user into an unrelated screen. Instead, it should show a safe fallback and provide a route to complete or correct the profile.

Profile editing should validate fields and should not allow a user to modify protected fields belonging to another account.

D.6 Blood Request Creation Use Case

Primary actor: requester. Preconditions: the request form is accessible and the user understands which information will be shared. Main flow: the requester opens the form, selects or enters the blood group, provides the required request details,

checks the information and submits it. The application validates required values and gives a progress indication while saving. A success confirmation should be shown only after the database confirms that the record was written.

Alternative flows include incomplete fields, invalid values, cancelled submission, duplicate submission caused by repeated tapping, loss of connectivity and permission denial. The user should be able to review the information before submission.

D.7 Donor Request Viewing Use Case

Primary actor: donor. Preconditions: the donor has reached the request-list screen and the application has permission to read the relevant records. Main flow: the screen requests records from the configured data source, displays a loading state, and then shows matching request cards. If no records are available, an empty-state message should explain that there are currently no matching entries.

If a read fails, the screen should show an error state and a retry action rather than an empty list that could be mistaken for a successful query. Any filters must be documented and tested so that valid requests are not hidden unintentionally.

D.8 Acceptance Criteria

The registration and login forms should reject missing mandatory values and show understandable validation messages. A successful login should lead to the intended screen. A request submission should not be reported as successful unless the data service confirms the write. The donor request list should distinguish loading, populated, empty and error states. A user should not be able to read or modify data outside the permissions granted by the configured security rules.

These criteria should be verified on the final build and the observed outcomes entered in the test record in Appendix G.

APPENDIX E – DATA DICTIONARY AND DATA-MANAGEMENT GUIDELINES

E.1 Introduction

A data dictionary defines the meaning and expected format of information used by an application. It helps the interface and backend use the same field names and reduces errors caused by inconsistent spelling, capitalisation or data types. The entries below are representative design guidance for a donor-request application. They are not confirmation of the exact collection names or schema in the current LifeLink build.

The final schema must be checked against the actual Firebase project or other configured data store.

E.2 User Account Information

A user record may contain a unique account identifier supplied by the authentication service, a display name, an email address, a role, and creation or update timestamps. The unique identifier should be used to associate profile data with the authenticated account. Email addresses should not be used as the sole database key because an email address can change and may not be suitable as a stable identifier. The role field should use a limited set of documented values.

A role should not be trusted merely because it is supplied by a client-side form; access control must be enforced by backend rules or trusted server-side logic.

E.3 Donor Profile Information

A donor profile may include the user identifier, blood group, broad location information, availability status if supported, and optional contact preferences. Only information required for the coordination task should be collected. The application should explain which profile fields can be seen by other users. Sensitive contact information should not be displayed publicly by default. If the project does not implement a verified-donor workflow, the report and interface must not label a profile as medically verified.

Blood group and availability are user-provided values unless a documented verification process exists.

E.4 Blood Request Information

A request record may contain a unique request identifier, the identifier of the user who created it, required blood group, units or quantity if the form collects it, location or hospital details if collected, a contact method, a request description, status and timestamps. The data model should define which fields are mandatory and which are optional. Date and time values should use a consistent representation. Free-text fields should have reasonable length limits.

A request status should use a small documented set of values rather than arbitrary text, if status tracking is implemented.

E.5 Data Validation

Validation should be applied at more than one level. The user interface can check whether required fields are empty, whether values have the expected format and whether a value exceeds a sensible length. Backend rules should separately control who can create, read, update or delete records. Client-side validation improves usability but is not a security boundary because a modified client can bypass it.

Error messages should describe how the user can correct the input without revealing internal database paths or credentials.

E.6 Data Consistency

The create operation and read operation must use the same collection or document path and the same field names. For example, if a request is stored under a field named bloodGroup, the display screen should not attempt to read blood_group unless an explicit mapping exists. The application should handle missing optional fields without crashing. If older records use a different schema, the team should document the migration or compatibility approach. Test data should be clearly distinguished from real user information.

E.7 Data Retention and Deletion

The project should define how long request and profile information is retained and how a user can ask for correction or deletion. The exact process depends on the final backend configuration and institutional requirements. Data should not be kept indefinitely without a reason. Before deleting a user account, the application design should consider whether related requests must be deleted, anonymised or retained for a documented purpose.

These policies should be reviewed with the project guide before deployment beyond a prototype.

E.8 Example Field Catalogue

The following field catalogue can be used as a review checklist: userId – unique account reference; displayName – name shown in the interface; email – account contact value; role – documented user category; bloodGroup – selected blood group; location – user-entered area or facility; requestId – unique request reference; requesterId – account reference of the creator; requestStatus – documented request state; createdAt – creation timestamp; updatedAt – latest update timestamp.

Confirm whether each field exists in the final implementation, and remove fields that are not used.

APPENDIX F – USER INTERFACE, ACCESSIBILITY AND ERROR-STATE GUIDELINES

F.1 Interface Design Objectives

The interface should help users understand the next action without requiring them to remember complex instructions. Important actions such as registration, login, creating a request and viewing requests should be easy to locate. Labels should use familiar terms and should remain consistent across screens. The same action should not have different names on different screens unless the meaning is genuinely different. The layout should remain readable on the screen sizes supported by the project.

F.2 Navigation and Screen Structure

Navigation should make the relationship between the dashboard, profile and request-related screens clear. A user should be able to identify the current screen and return to the previous screen without losing entered information unexpectedly. Destructive actions, if present, should require confirmation. The application should avoid dead-end screens. If a feature is unavailable in the current prototype, the interface should not present a control that appears functional but does nothing.

F.3 Forms and Input Controls

Forms should group related information and use labels that remain visible while the user enters values. Required fields should be identified clearly. Input controls should match the type of information: email keyboards for email addresses, numeric input for numeric values, and selection controls for a fixed list such as blood groups. The application should preserve user input when a recoverable error occurs, where practical.

A submit button should show a progress state during a network operation and should prevent repeated submissions while the first operation is pending.

F.4 Colour and Contrast

Colour can help distinguish primary actions, secondary information and status messages, but it should not be the only way to communicate meaning. Text should have sufficient contrast against its background in both light and dark themes. Error messages should include text or an icon as well as colour. The team should review all screens for low-contrast text, disabled controls that are difficult to read and colour combinations that may be confusing to users with colour-vision differences.

F.5 Text and Readability

Use concise labels, familiar words and consistent text sizes. Avoid long paragraphs inside small cards. Important information should be visually prioritised without relying on decorative effects. Text scaling should be checked where the target platform supports it. The interface should not clip labels or hide buttons when text becomes larger. Alignment and spacing should be consistent so that users can scan information quickly.

F.6 Loading, Empty and Error States

Cloud-backed screens should account for delays and failures. A loading state communicates that the application is working. An empty state communicates that the request completed but there is no data to show. An error state communicates that the application could not complete the operation. These states must not be confused. Error messages should offer an appropriate next step, such as retrying, checking the connection or reviewing entered fields.

Technical details should be recorded in development logs where appropriate, not exposed to ordinary users.

F.7 Accessibility Review Checklist

Check that every input has a meaningful label, buttons have understandable names, text contrast is adequate, information is not conveyed by colour alone, touch targets are large enough, and navigation order is logical. Check that icons have accessible descriptions where required. Review the app using larger text settings and different screen sizes. If screen-reader testing has not been completed, record it as pending rather than claiming accessibility compliance.

F.8 User Feedback and Iteration

Feedback should be collected with a consistent set of questions. Ask participants whether they could identify the main actions, understand the request fields, interpret the information displayed and recover from an error. Record suggestions separately from changes actually implemented. Do not publish names, phone numbers, email addresses or other identifying details in the report. A small informal feedback exercise can reveal usability issues, but it should not be described as a statistically representative study.

APPENDIX G – EXPANDED SOFTWARE TEST PLAN AND RECORD SHEETS

G.1 Purpose

This appendix provides a test plan for checking the main LifeLink workflows. It is a test template, not a record of tests already completed. The project team should execute each relevant case against the final build, record the actual result, and mark the outcome as Pass, Fail, Blocked or Not Applicable. Screenshots or logs may be retained as evidence, with personal information and credentials removed.

G.2 Test Environment Record

Before testing, record the application version or build identifier, test date, device or browser model, operating system version, network condition, backend environment and test account type. Record whether the test used a development or production backend. Never include passwords, private keys, service-account files or unrestricted database credentials in the report. If different devices are used, keep a separate environment record for each device.

G.3 Registration Test Scenarios

Test a valid registration using a controlled test account. Test missing name, missing email, invalid email format, missing password and password confirmation mismatch if the form includes confirmation. Test an email address that already has an account. Test behaviour when the network is disconnected before submission and when connectivity is lost during the request. Confirm that the user sees an accurate result and that no duplicate profile is created by repeated tapping.

Record the actual message and resulting screen.

G.4 Login Test Scenarios

Test login with valid credentials, an incorrect password, an unknown email, empty fields and network unavailability. Confirm that the password is masked by default and that the user can reveal it only through an explicit control if such a control is provided. Check that a failed login does not navigate to a protected screen. Check

that a successful login restores the intended session behaviour. If sign-out is supported, test sign-out and confirm that protected screens cannot be accessed after the session ends.

G.5 Profile and Role Test Scenarios

Check that the correct profile is displayed for the signed-in account. Check behaviour when profile data is missing, incomplete or contains optional fields that are blank. If donor and requester roles are implemented, test each role separately and verify the intended navigation. Attempt to access another user's profile or data using a test account with insufficient permission. The expected outcome is that access is denied by the backend security configuration, not merely hidden by the interface.

G.6 Blood Request Test Scenarios

Submit a request with valid test values and confirm the saved record through the application or authorised backend inspection. Test missing required fields, unsupported blood-group values, unusually long text, repeated submission and a network failure. Confirm that the form does not show success when the write fails. Check whether the request appears in the correct list and whether the displayed details match the submitted values. Use fictional test information only.

G.7 Donor List Test Scenarios

Check the list when matching records exist, when no records exist and when the database read fails. Verify that the loading indicator disappears after completion. Check the ordering and filters if they are implemented. Confirm that the query does not unintentionally exclude valid records because of inconsistent field names, role values or case sensitivity. Test long request descriptions and missing optional fields so that the list remains usable.

G.8 Security and Privacy Test Scenarios

Review database rules for unauthorised reads and writes. Confirm that users can update only the records they are permitted to update. Verify that secrets are not

stored in source code or exposed in screenshots. Check that personal information is not unnecessarily displayed in request cards. Confirm that logs and test evidence do not contain passwords or private tokens. Security testing should be performed only in an authorised development environment.

G.9 Results Recording Template

For each test, record: Test ID; feature; preconditions; input data; steps; expected result; actual result; status; evidence reference; tester initials; and remarks. If a test fails, record the observed behaviour, steps to reproduce, severity, and whether the issue was corrected and retested. A test should not be marked Pass based only on the screen appearing; the relevant data operation and permission behaviour should also be checked.

G.10 Defect Classification

A critical defect prevents a core workflow or exposes protected information. A major defect causes a key function to fail without a reasonable workaround. A minor defect affects presentation or a secondary action but does not block the main workflow. An observation is an improvement suggestion rather than a failure against a stated requirement. The project team should use a consistent classification and avoid exaggerating the severity of cosmetic issues.

APPENDIX H – INSTALLATION, CONFIGURATION AND USER GUIDANCE

H.1 Scope of This Guide

This guide describes a general approach for preparing and using a Flutter application that depends on Firebase. Exact commands, configuration files and screen names must be checked against the actual project repository. The guide should be updated if the final build uses a different backend or platform. It does not include private configuration values or credentials.

H.2 Development Prerequisites

A development computer requires the Flutter SDK, a supported code editor, the Dart tooling bundled with Flutter, and the platform tools needed for the selected target. The Flutter installation should be checked using the diagnostic command supplied in the official Flutter documentation. The project should be opened from the directory containing its pubspec.yaml file. Dependencies should be restored from the project manifest so that the team uses the declared package versions.

H.3 Firebase Configuration Considerations

If Firebase is used, the application must be registered with the correct Firebase project and platform identifiers. Platform configuration files should correspond to the application identifier used by the build. Authentication providers must be enabled in the Firebase console if the application relies on them. Database rules should be configured before testing data operations. Configuration files and API values should be handled according to the provider’s security guidance.

Private service-account credentials must never be embedded in a client application.

H.4 General Build Workflow

A typical workflow is to open the project root, restore dependencies, review static analysis messages, run the application in a development environment, and execute the test plan. The team should confirm that the intended platform is selected before creating a release build. The release should be tested separately from the

development build because configuration and permission differences can affect behaviour.

Build commands should be copied from the project's actual platform setup rather than assumed from a generic example.

H.5 User Login

A user opens the application and selects the login option. The user enters the registered account details and submits the form. The application should provide clear feedback while the request is processed. If login succeeds, the user is directed to the appropriate dashboard. If login fails, the user should check the entered details or connection and retry. The application should not ask users to share their passwords with project team members.

H.6 Creating a Blood Request

A requester opens the request form and reads the field labels carefully. The requester enters only accurate information and checks the details before submitting. The user should understand which details may be visible to donors. After submission, the application should show confirmation only when the save operation succeeds. If the application reports an error, the user should avoid repeatedly submitting the same request until the status is clear.

H.7 Viewing Donor Requests

A donor opens the relevant request list and reviews the information shown. The donor should independently assess whether they are able and willing to respond. The application does not determine medical eligibility, guarantee compatibility or confirm that blood is available. Users should follow the procedures of recognised blood banks and healthcare professionals. The app is a coordination aid and must not be treated as medical advice.

H.8 Troubleshooting

If the application does not open, confirm that the correct build is installed and that the device meets the platform requirements. If login fails, check connectivity and

the account details without sharing credentials. If a submitted request is not visible, the development team should check the write result, collection path, field names, query conditions and security rules. If a screen remains in a loading state, check asynchronous error handling and network connectivity.

Capture a screenshot of the issue only after removing personal data.

H.9 Maintenance

Maintenance includes updating dependencies, reviewing security rules, correcting defects, checking compatibility with platform updates and reviewing user feedback. Changes to the data schema should be planned and tested because existing records may use older field formats. The team should keep a record of build versions and important configuration changes. A maintenance release should be tested using the same core test cases as the initial build.

APPENDIX I – PROJECT RISK ANALYSIS AND MITIGATION PLAN

I.1 Introduction

Risk analysis identifies events that may affect project completion, application reliability, data protection or user experience. A risk should be described in terms of its cause, possible effect and planned response. The table and descriptions below are planning guidance. The team should review the likelihood and impact values based on its actual development experience and update the record during project reviews.

I.2 Authentication Risk

Incorrect authentication configuration may prevent users from registering or logging in. The risk can be reduced by verifying the selected provider, platform configuration and error handling in a controlled test environment. The team should test both successful and unsuccessful authentication paths. Credentials must not be stored in plain text or included in screenshots. A failed login should not accidentally expose a protected screen.

I.3 Database Integration Risk

The application may write data to one collection or document path while the display screen reads from another. Inconsistent field names or data types can also cause records to disappear from a query. Mitigation includes maintaining a documented data dictionary, reusing constants for field names, checking write acknowledgements and testing reads against known test records. Database rules should be reviewed separately from UI logic.

I.4 Privacy and Security Risk

Donor and requester information may include personal or health-adjacent details. Excessive data collection or broad database permissions can expose users to unwanted contact or misuse. Mitigation includes data minimisation, clear visibility notices, authenticated access, restrictive security rules, safe handling of logs and removal of personal details from report screenshots. Any real-world deployment requires a more complete privacy and security review.

I.5 Usability Risk

Users may misunderstand a form, overlook an important field or misinterpret a status message. Mitigation includes clear labels, consistent layouts, field validation, readable contrast and feedback from representative users. The team should record observed usability issues and distinguish them from suggestions that have not yet been implemented.

I.6 Connectivity Risk

Cloud operations depend on network access. Poor connectivity can interrupt sign-in, request submission or list retrieval. The interface should show progress and failure states and should avoid reporting a successful save before confirmation. Retry behaviour should be designed to avoid creating duplicate records. Offline support should not be claimed unless it has been implemented and tested.

I.7 Schedule and Scope Risk

A student project may face limited time for integration, testing and documentation. The team should prioritise core workflows, track unfinished tasks and avoid adding optional features before essential flows are stable. A feature should be marked complete only after implementation and testing. The report should state limitations honestly rather than describing planned work as completed.

I.8 Risk Review Procedure

At each project review, the team should list open risks, assign an owner, estimate likelihood and impact, record mitigation actions and set a review date. When a risk occurs, record the actual issue and the action taken. If a mitigation is unsuccessful, revise the plan. Risk tracking is useful throughout development and should continue during testing and final preparation.

APPENDIX J – GLOSSARY, ABBREVIATIONS AND PROJECT REVIEW CHECKLIST

J.1 Glossary

Authentication: the process of confirming the identity of a user. Authorisation: the process of deciding what an authenticated user is allowed to access. Backend: the services and data operations that support the application. Cloud database: a remotely hosted data store accessed through a network. Donor: a person who may be willing to donate blood, subject to appropriate medical assessment. Requester: a person who enters a blood-related request.

User interface: the screens and controls through which a person interacts with the application. Validation: checking that entered information meets defined rules. Test case: a documented set of preconditions, steps, inputs and expected outcomes used to verify behaviour.

J.2 Technical Terms

Flutter: a software development toolkit for building applications using Dart. Dart: the programming language used by Flutter. Firebase Authentication: a service that provides account authentication. Cloud Firestore: a cloud-hosted NoSQL document database. Widget: a building block used to compose a Flutter user interface. Asynchronous operation: an operation that may complete later, such as a network request. CRUD: create, read, update and delete operations.

Responsive layout: a layout that adapts to different screen sizes and orientations.

J.3 Review of Report Accuracy

Before submission, compare every feature described in the abstract, objectives, scope, methodology, results and conclusion with the actual application. Remove statements about functions that are not present. Confirm whether the project uses Firebase Authentication and which database service is configured. Confirm the exact names of user roles and screens. Check that screenshots show the final build

and do not expose personal information. Replace all bracketed placeholders with accurate information.

J.4 Review of Technical Evidence

For each core workflow, retain evidence that supports the report: screenshots of the relevant screens, test records, non-sensitive database observations, build information and defect notes. Evidence should correspond to the version described in the report. A screenshot alone may demonstrate the appearance of a screen but does not prove that authentication or database operations work. Pair visual evidence with test records where possible.

J.5 Review of References

Check every reference against the original official documentation or publication. Include only sources actually consulted by the team. Confirm the document title, organisation or author, publication details where available and access date if required by the department. Remove conditional references that do not apply to the final implementation. Do not leave instructions such as “add verified papers” in the submitted reference list.

J.6 Final Submission Checklist

Confirm that the cover and title pages contain the correct project title, team names, register numbers, department, institution and submission period. Obtain the required signatures on the bonafide certificate. Check the order of preliminary pages, chapter numbering, table and figure numbering, captions, page numbers and table of contents. Verify that all tables and figures are cited in the text. Check spelling, grammar, margins, font and line spacing.

Confirm that the PDF opens correctly and that the editable document is retained by the team.

J.7 Project Review Readiness

During the project review, each team member should be able to explain the problem statement, objectives, architecture, technology choices, main modules, data flow,

testing approach, limitations and future work. Team members should be clear about which features are implemented and which remain planned. Demonstrations should use test accounts and fictional data. If a feature fails during demonstration, explain the observed behaviour accurately and describe the debugging steps without claiming an unverified result.

REFERENCES

Dart, “Dart Language Documentation.” Available at: https://dart.dev/guides (accessed on [insert date]).

Firebase, “Cloud Firestore Documentation.” Available at: https://firebase.google.com/docs/firestore (use this reference only if Cloud Firestore is used).

Firebase, “Firebase Authentication Documentation.” Available at: https://firebase.google.com/docs/auth (accessed on [insert date]).

Firebase, “Firebase Documentation.” Available at: https://firebase.google.com/docs (accessed on [insert date]).

Flutter, “Flutter Documentation.” Available at: https://docs.flutter.dev/ (accessed on [insert date]).

Add the verified research papers, articles or official guidance actually consulted by the team.

Add any institutional project guidelines or approved reference material required by the department.